📘 MODULE 29: CORRELATION SEARCHES

29.1 Concept

A **correlation search** is a **scheduled saved search that detects a pattern** across events and creates an alert-like record. In **Splunk Enterprise Security (ES)** it is a specific object type that can create **notable events**, add **risk**, and run **adaptive response actions**.

Correlation = combining multiple signals, for example "many failures **followed by** a success from the same IP."

29.2 Relationship to ES

- In ES, correlation searches run on a schedule against **CIM data models** (using `tstats`).
- When the logic matches, the correlation search outputs a **notable event** (a work item in Incident Review) and/or **risk modifiers**.
- **ES is a separate, licensed product.** Without it, you build the same logic using a normal saved search or alert.

29.3 Detection logic without ES

Detection 1: brute force then success (raw data)

spl

```
index=linux_auth sourcetype=linux_secure ("Failed password" OR "Accepted password") earli
| rex "(?<outcome>Failed|Accepted) password for (?:invalid user )?(?<user>\S+) from (?<sr
| stats count(eval(outcome="Failed")) AS failures,
        count(eval(outcome="Accepted")) AS successes,
        min(eval(if(outcome="Failed",_time,null()))) AS first_fail,
        max(eval(if(outcome="Accepted",_time,null()))) AS last_success,
        values(user) AS users by src_ip
| where failures>=10 AND successes>=1 AND last_success>first_fail
```

What it does: per source IP, counts failures and successes in 30 minutes and keeps IPs with at least 10 failures and a success that came after the first failure.
Line-by-line:

- The base search keeps only the two relevant message types.
- `rex` extracts outcome, user, and IP in one pass.
- `count(eval(condition))` is a **conditional count**.
- `min/max(eval(if(...,_time,null())))` find the time of the first failure and the latest success.
- `where` applies the detection logic.

Common mistakes: `rex` not matching all variants, 30-minute window vs 15-minute schedule overlap (use throttle), using `=` instead of `==` in `if()`.
Interview explanation: "I use one `stats` with conditional counts instead of join or transaction, so it's one pass and scales."

Detection 2: password spraying (many users, few attempts)

```spl
index=linux_auth sourcetype=linux_secure "Failed password" earliest=-30m
| rex "Failed password for (?:invalid user )?(?<user>\S+) from (?<src_ip>\d{1,3}(?:\.\d{1,
| stats count, dc(user) AS users by src_ip
| where users>=10 AND count/users<=3
```

Logic: many different accounts, few tries each. Thresholds must be tuned.

Detection 3: CIM version (what ES uses)

```spl
| tstats summariesonly=true count, dc(Authentication.user) AS users
  FROM datamodel=Authentication
  WHERE Authentication.action="failure"
  BY Authentication.src
| rename Authentication.* AS *
| where count>=10
```

Works across **all** CIM-mapped sources (Linux, Windows, VPN), which is the point of CIM.

Detection 4: silent host

Use the `tstats` search from Module 15.

29.4 Turning detections into alerts

1. Save the search as a scheduled alert (Module 28), with the time range equal to the schedule (e.g. 15 min). If the logic needs a longer window (30 min), schedule every 15 minutes and **throttle by** `src_ip`.
2. Add **context fields** to the results (user, count, time range) so triage is easy.
3. Use a **lookup allowlist** to remove known-good scanners:

```spl
  | lookup scanner_allowlist.csv ip AS src_ip OUTPUT approved
  | where isnull(approved)
```

4. Document: purpose, data source, logic, threshold, known false positives, response steps.

29.5 What makes a good correlation search

Property	Why
Specific time window	Performance and precision
Grouped by an entity (`src`, `user`, `dest`)	One alert per entity
Tuned threshold	Reduces false positives
Allowlist/exceptions	Known-good activity doesn't alert
Uses CIM + `tstats` where possible	Fast and portable
Documented	Analysts know what to do

29.6 Hands-on

1. Run Detection 1 with a small threshold (`failures>=3`) after generating failed SSH attempts against your lab VM (a few wrong passwords from another terminal or machine).
2. Convert it into a scheduled alert (every 15 minutes, throttled by `src_ip`).
3. Add a lookup allowlist for your own IP.

29.7 Troubleshooting

Symptom	Cause
Never fires	Threshold too high, time window shorter than the pattern, regex fails
Fires constantly	Threshold too low, no allowlist, no throttle
Misses events	Window/schedule gap, or data arrives late
CIM version returns nothing	Model not accelerated or data not mapped (Module 17)

29.8 Interview Q&A

- **What is a correlation search?** A scheduled search that detects patterns across events and creates a notable event or alert.
- **How does it relate to ES?** ES correlation searches run against CIM data models and create notables, risk, and response actions.
- **Brute force detection logic?** Count failures per source over a window, threshold, optionally followed by a success.
- **How do you avoid false positives?** Tuning, allowlists, throttling, grouping by entity.
- **Why `stats` and not `join`?** One pass, no subsearch limits.

29.9 Revision

Correlation = scheduled pattern detection · ES creates notables/risk · same logic works as plain alerts · group by entity · conditional `count(eval())` · tune + allowlist + throttle.